

OPERATING SYSTEMS

Chrono-Rift Game



NATIONAL UNIVERSITY OF COMPUTER & EMERGING
SCIENCES ISLAMABAD CAMPUS

Group Info

MEMBER 01: M. Aashir Sheraz

ROLL NO: 23i-0025

MEMBER 02: M. Yasir Suhail

ROLL NO: 23i-0566

Table of Contents

1. Introduction.....	3
2. Process Architecture	3
3. Shared Memory and IPC.....	3
4. Threading Model.....	4
4.1 Arbiter Threads	4
4.2 HIP Threads	4
4.3 ASP Threads	4
5. Turn Scheduling (Stamina System)	5
6. Stun Mechanism.....	5
7. Inventory System	5
8. Artifacts and Deadlock Detection	6
8.1 Artifacts.....	6
8.2 Deadlock Detection and Resolution.....	6
9. Ultimate Ability	6
10. Game Completion Conditions.....	7
11. Stat Formulas	7
12. Weapon Catalogue	7
13. Bugs Found and Fixed	8
14. Screenshots	8

1. Introduction

Chrono Rift is a turn-based battle game developed as part of Operating Systems course. The purpose of this project was to apply core OS concepts such as processes, threads, shared memory, semaphores, and signals in a real working program. The game runs as three separate processes working together and communicates purely through shared memory, with no pipes or message queues used anywhere.

This report explains how each part of the system was designed and implemented, what problems came up during development, and how those problems were fixed. The roll number used for all formulas is 25.

2. Process Architecture

The system runs as three completely separate processes. Each one has its own job and they coordinate through a shared memory region.

The Arbiter (arbiters) is the main process and the brain of the whole game. It creates the shared memory, waits for the other two processes to connect, then runs the game loop. It handles turn scheduling, damage calculation, wave spawning, artifact management, deadlock detection, and it also runs the SFML graphical window. It is the only process that is allowed to modify game state.

The Human Interfacing Process (hips) is responsible for reading player input from the keyboard. It spawns one dedicated thread per player. Whichever player's turn it is, that thread reads the input and writes it to the shared move buffer. The other threads just wait. This process never directly changes any game state — it only writes to the move buffer and lets the Arbiter handle the rest.

The Automated Strategic Process (asps) handles all enemy AI. It spawns one dedicated thread per enemy. Each enemy thread waits for its turn, then decides whether to strike, skip, or try to lock an artifact. On each new wave, old threads are joined and fresh threads are created for the new enemies.

3. Shared Memory and IPC

All communication between the three processes happens through a single shared memory segment named `/chrono_rift_game`. This was created using `shm_open` and `mmap`. The shared memory holds everything including player and enemy states, the move buffer, inventory,

weapons, artifacts, action log, and all the semaphores. No pipes, FIFOs, or message queues were used anywhere, as required by the project specification.

The SharedState structure is mapped at the same virtual address in all three processes. Because all three processes read and write this structure, access is controlled using semaphores. The main semaphore is `state_lock` which works like a mutex. Any code that reads or writes shared state must first call `sem_wait` on it and then `sem_post` when done.

All semaphores are unnamed and initialized with `pshared=1`, which means they work across processes. The Arbiter is the only one that calls `sem_init` to initialize them, and it also calls `sem_destroy` when the game ends.

4. Threading Model

4.1 Arbiter Threads

The Arbiter runs several threads besides the main game loop. A dedicated render thread reads the shared state and updates the SFML window continuously and independently from the game logic. A deadlock monitor thread runs in the background every 0.6 seconds checking for circular waits between artifacts. A signal thread handles `SIGINT`, `SIGTERM`, and `SIGALRM` without interrupting the main loop. Whenever a player stuns an enemy, a short-lived stun-clear thread is spawned that sleeps for exactly 3 seconds and then clears the stun.

4.2 HIP Threads

The HIP spawns one pthread for each human player, so between 1 and 4 threads depending on party size. Only the thread whose player currently has a full stamina bar and is waiting for input will be active. All other threads sit in an idle loop sleeping 100ms at a time. This satisfies the project requirement of one dedicated thread per player.

4.3 ASP Threads

The ASP spawns one pthread per enemy. On wave 1 it creates threads for all enemies in that wave. When all enemies from a wave die, the ASP detects this, joins all the old threads, resets its counter to zero, and then spawns fresh threads for the next wave. This was an important bug fix because the original code never reset the counter, so wave 2 enemies never got threads and the AI was completely dead from wave 2 onward.

5. Turn Scheduling (Stamina System)

Turns are not round-robin. Instead, every 0.1 seconds the Arbiter adds each entity's speed value to their stamina. The first entity whose stamina reaches or exceeds their maximum gets to act. After acting, their stamina resets to zero. This means faster entities act more often than slower ones.

Player speed is calculated as 100 divided by the number of players. Enemy speed is a random number between 10 and 30. With many enemies in a wave, it is possible for multiple enemies to have full stamina at the same time, in which case the one with the highest stamina value goes first.

If a player's turn comes and no input is received within 3 seconds, the Arbiter automatically skips that turn and sets their stamina to 50 percent of maximum. This prevents the game from freezing when a player is not paying attention. Stunned and dead entities are skipped entirely during stamina accumulation.

6. Stun Mechanism

The stun mechanism was designed to meet the project requirement of pausing an enemy without the target process actively polling a flag. The implementation works as follows.

When a player stuns an enemy, the Arbiter sets `enemy_stunned[i]` to 1 in shared memory and then sends SIGUSR1 to the ASP process. This signal is the asynchronous notification required by the specification. At the same time, the Arbiter spawns a stun-clear thread that sleeps for exactly 3 seconds.

On the ASP side, each enemy thread checks the stun flag at the start of its loop. If the flag is set, the thread calls `sem_wait` on its dedicated `enemy_stun_sem` semaphore. This call blocks the thread completely with zero CPU usage as there is no flag polling, no spinning, no usleep loop. After exactly 3 seconds, the Arbiter's stun-clear thread clears the flag and calls `sem_post` on that semaphore, which wakes the enemy thread. This is a true non-blocking interrupt as required.

Each enemy has its own dedicated stun semaphore (`enemy_stun_sem[0]` through `enemy_stun_sem[8]`) stored in shared memory and initialized with `pshared=1` so it works across processes.

7. Inventory System

The inventory is an array of 20 integer slots. Each weapon occupies a certain number of contiguous slots. Solar Core takes 10 slots, Lunar Blade takes 10 slots, Iron Halberd takes 7

slots, Thunderstaff and Frostbow take 6 slots each, Obsidian Axe takes 5 slots, Venom Dagger takes 4 slots, and Splinter Stick takes only 2 slots. A continuation marker value of -2 is used to fill the extra slots a weapon occupies beyond its first.

When a player tries to pick up a weapon and there is enough contiguous space, it is placed directly. If there is not enough room, the system evicts the cheapest weapons by slot cost to a long-term storage area (maximum 10 items) until space is freed. This means only the minimum necessary weapons are moved out. Weapons in storage can be swapped back in using action 11.

When an enemy dies there is a 55 percent chance of dropping a random weapon on the ground. If no player picks it up within 10 ticks (approximately 6 seconds), an enemy automatically picks it up. Weapons held by enemies are never dropped when that enemy dies.

8. Artifacts and Deadlock Detection

8.1 Artifacts

There are three artifacts. Solar Core and Lunar Blade are available from the start. The Eclipse Relic spawns automatically once 3 enemies have been killed total, and it appears in the arena for anyone to lock. Each artifact has lock ownership and waiting fields stored in shared memory.

A critical bug in the original code caused the Eclipse Relic to be completely non-functional. The `copy_text` call that sets the relic's name was accidentally placed after a comment marker, making it dead code. Because `handle_artifact_lock` checks if `name[0]` is null before doing anything, the relic could never be locked or used. This was fixed by moving the `copy_text` call to its own line before the early return.

8.2 Deadlock Detection and Resolution

The deadlock monitor thread runs every 0.6 seconds and scans all artifact pairs looking for a circular wait condition. A circular wait means Entity A holds artifact X and is waiting for artifact Y, while Entity B holds artifact Y and is waiting for artifact X. When this is detected, the entity with lower HP is chosen as the victim and all artifact locks it holds are released. This is logged in the action log so it is visible in the game.

9. Ultimate Ability

The Ultimate ability is available to any player who has both Solar Core and Lunar Blade simultaneously in their active inventory. When triggered, the Arbiter sends SIGSTOP to the ASP process. SIGSTOP cannot be caught or ignored, it immediately suspends all threads in the ASP, which means all enemies freeze completely.

At the same time, the Arbiter calls alarm(10), which schedules a SIGALRM signal after 10 seconds. When SIGALRM arrives, the signal thread in the Arbiter sends SIGCONT to the ASP process, which resumes it. The entire mechanism uses only signals, no flags, no pipes, no polling of any kind.

10. Game Completion Conditions

There are three ways the game can end. If the total enemies killed across all waves reaches 10, the players win and the message PLAYERS WIN is shown. If all players die at any point, the enemies win. If the player presses Ctrl+C at any time, the HIP sends SIGTERM to the Arbiter which then shuts the game down cleanly.

When all enemies in a wave die but the kill count is below 10, a new wave is spawned immediately. The Arbiter randomizes a new set of 2 to 9 enemies, resets their stats, and logs the new wave. The ASP then detects the wave change and creates new threads for the new enemies.

11. Stat Formulas

All random number generation is seeded with srand(25) in the Arbiter only. HIP and ASP do not seed the random number generator. The formulas below are derived from roll number 25.

Stat	Formula	Value
Player HP	25 + rand() in range 100 to 1000	Varies per player
Player Damage	Last digit of 25 (which is 5) + 10	15
Player Speed	100 divided by number of players	Varies
Enemy HP	25 + rand() in range 50 to 200	Varies per enemy
Enemy Damage	Second last digit of 25 (which is 2) + 10	12
Enemy Speed	rand() in range 10 to 30	Varies per enemy
Random seed	Roll number	25

12. Weapon Catalogue

The following table lists all 8 weapons, their inventory slot cost, and their damage value. These are fixed values defined in the weapon catalogue and do not change during the game.

Weapon Name	Inventory Slots	Damage
Solar Core	10	95
Lunar Blade	10	90

Weapon Name	Inventory Slots	Damage
Iron Halberd	7	55
Thunderstaff	6	50
Frostbow	6	48
Obsidian Axe	5	45
Venom Dagger	4	30
Splinter Stick	2	12

13. Bugs Found and Fixed

Several bugs were identified and fixed during the development process.

The first and most critical bug was the Eclipse Relic name never being set. The `copy_text` call was accidentally written on the same line as a comment, so the compiler ignored it. The relic would appear in logs as spawned but could never actually be interacted with because `handle_artifact_lock` checks for a non-null name before doing anything.

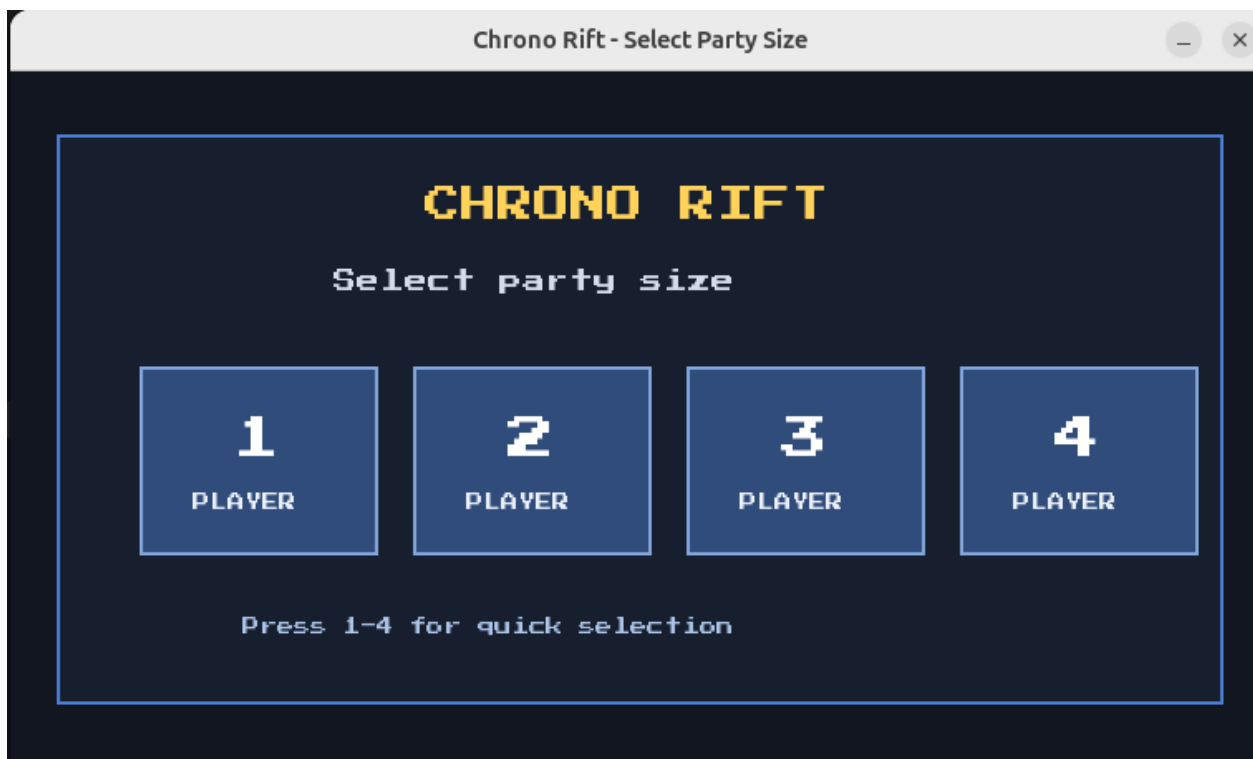
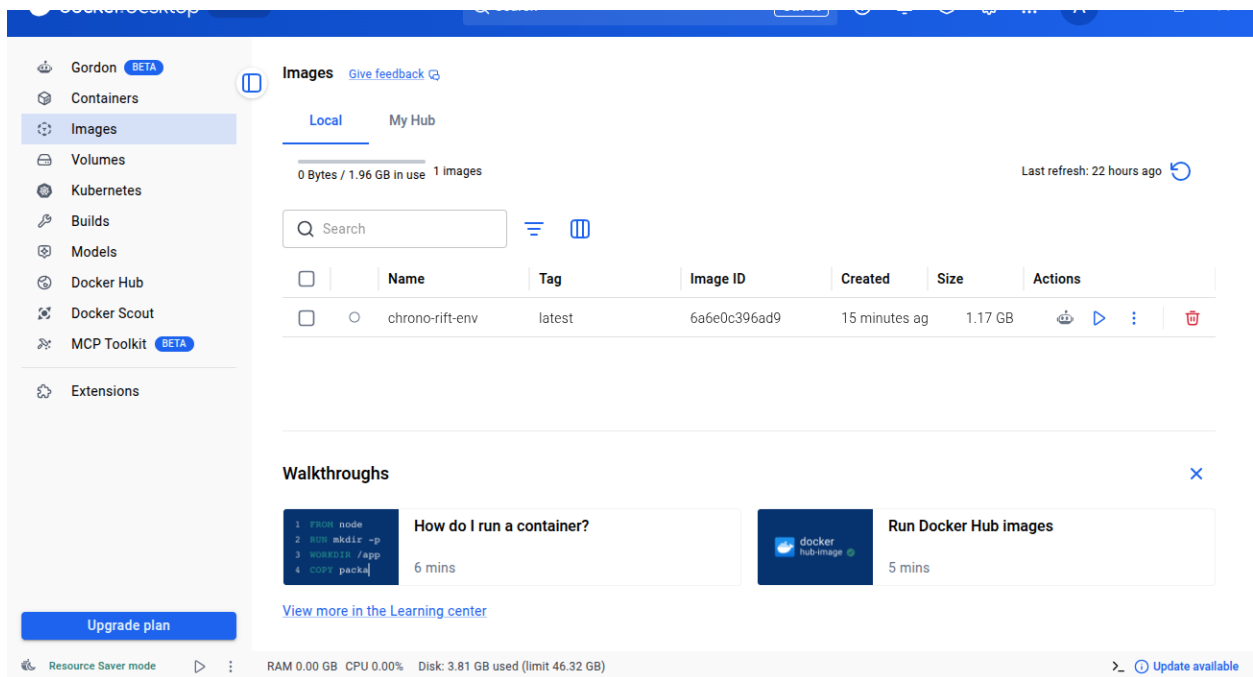
The second major bug was that enemy threads for wave 2 and beyond were never created. The ASP used a spawned counter that only ever increased. After wave 1 ended, the `check_active > spawned` would be false for any subsequent wave because spawned never reset. The fix was to detect when all enemies from the previous wave have died, join their threads, and reset spawned to zero so fresh threads are created for the new wave.

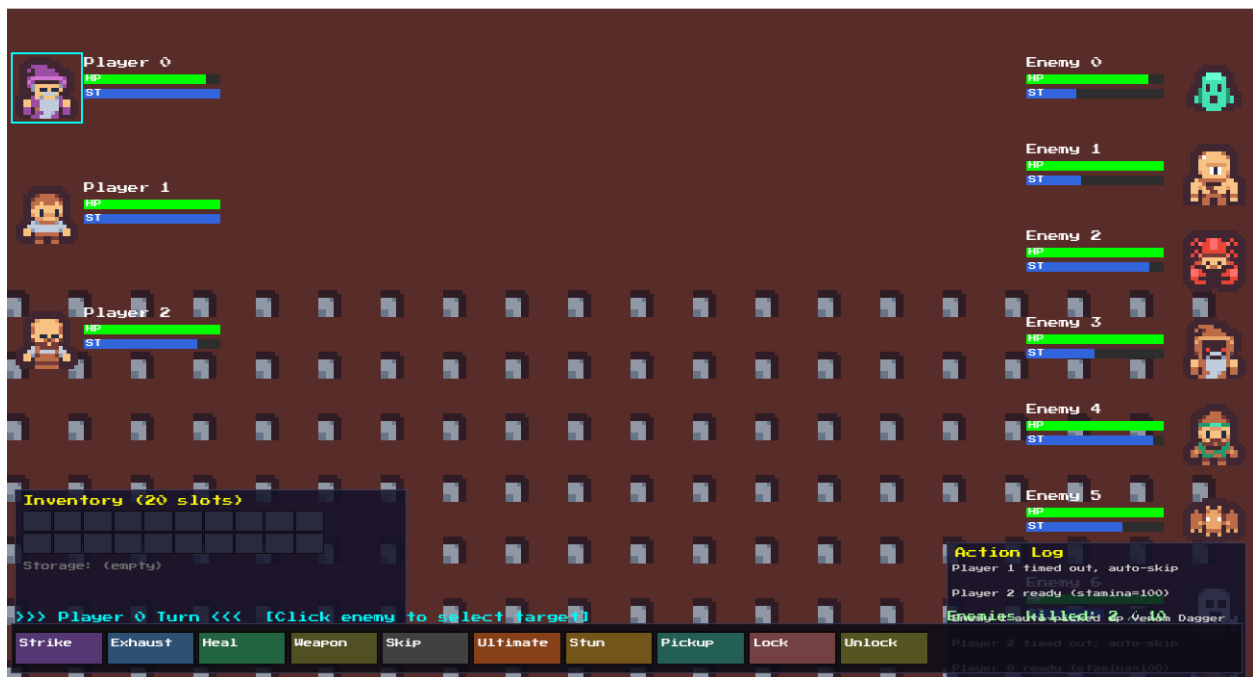
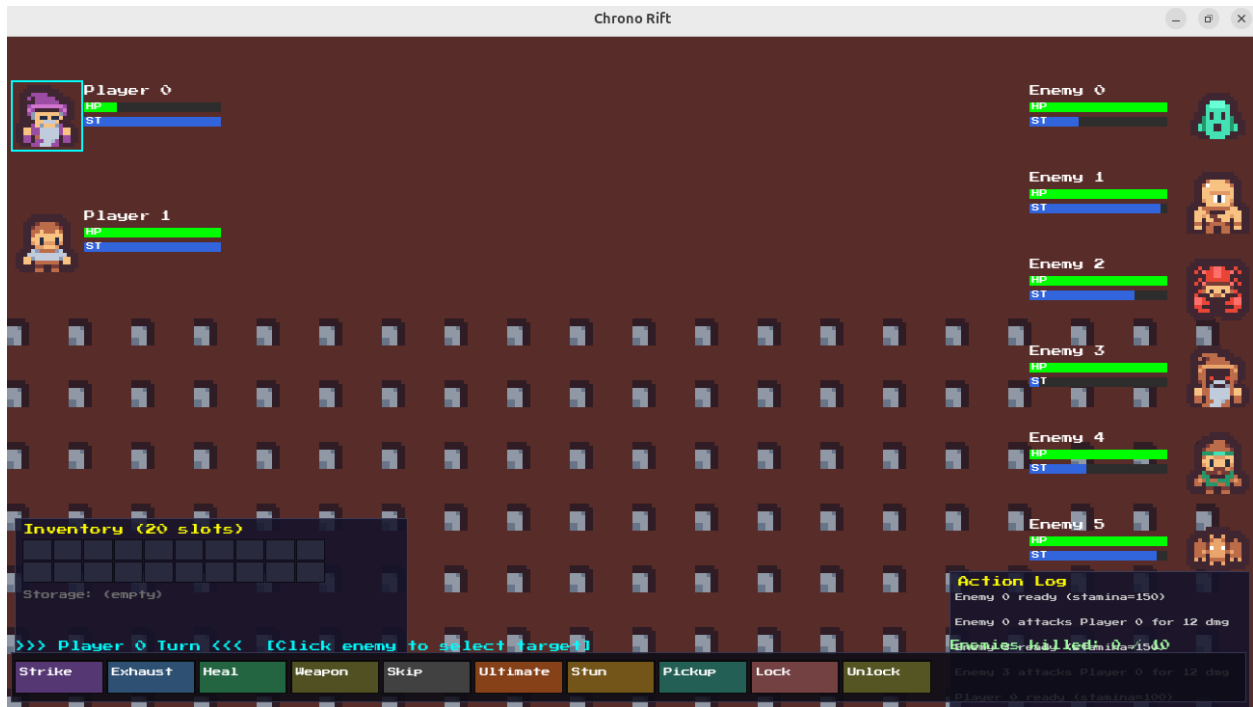
The third issue was that the stun mechanism used flag polling, which the specification explicitly forbids. The enemy thread would loop calling `usleep` and checking the flag continuously. This was replaced with a proper semaphore-based blocking mechanism where the enemy thread calls `sem_wait` and blocks completely until the Arbiter posts the semaphore after 3 seconds.

The fourth issue was a segmentation fault when the game ended with enemies winning. The Arbiter was calling `munmap` on the shared memory segment while ASP enemy threads were still reading from it. This was fixed by adding a 2-second sleep before destroying shared memory, and by adding early exit guards in ASP threads that check running before accessing any shared memory.

14. Screenshots

The following are some of screenshots from the game:





```
aashir-awan@aashir-awan-HP-348-G4:~/1/BCSD_23i-0025_AashirSheraz_23i-0566_YasirSuhail/BCSD_23i-0025_AashirShe
raz_23i-0566_YasirSuhail$ ./arbiters
[arbiter] Enemy 1 ready (stamina=150)
[arbiter] Enemy 1 attacks Player 1 for 12 dmg
[arbiter] Enemy 3 ready (stamina=150)
[arbiter] Enemy 3 skipped (stamina=50%)
[arbiter] Player 1 ready (stamina=100)
[arbiter] Player 1 strikes Enemy 0 for 15 dmg
[arbiter] Enemy 2 ready (stamina=150)
[arbiter] Enemy 2 attacks Player 1 for 12 dmg
[arbiter] Enemy 0 ready (stamina=150)
[arbiter] Enemy 0 attacks Player 1 for 12 dmg
[arbiter] Player 1 defeated!
[arbiter] === ENEMIES WIN ===
[arbiter] shutdown complete
aashir-awan@aashir-awan-HP-348-G4:~/1/BCSD_23i-0025_AashirSheraz_23i-0566_YasirSuhail/BCSD_23i-0025_AashirShe
raz_23i-0566_YasirSuhail$
```

15. Conclusion

This project gave us a practical understanding of how operating system concepts work together in a real application. Implementing three separate processes that communicate through shared memory, managing threads carefully to avoid race conditions, using signals for asynchronous communication, and designing a semaphore-based stun system were all challenging but educational experiences.

The most difficult parts were the Eclipse Relic bug which was very hard to spot because the `copy_text` call looked like it was there but was actually inside a comment, and the wave 2 threading bug where enemies silently stopped having AI without any obvious error. Both required careful reading of every line of code to find.

The final submission compiles cleanly with no warnings, runs correctly through multiple waves, handles all player actions, and shuts down without any crashes or segmentation faults.
